Reduced Weight Update Frequency for High Throughput Hardware Accelerator of the CCSDS-123.0-B-2 Algorithm

Aasmund Gabestad Nørsett

19/12/2025

Supervised by Milicia Orlandic and Samuel Boyle

Abstract

The HYPerspectral Smallsat for Ocean observation project at the Norwegian University of Science and Technology uses hyperspectral images for ocean observation. These images contain 120 spectral bands, resulting in large amounts of data that must be downlinked using limited bandwidth. In order to reduce data size, the algorithm CCSDS-123.0-B-2 is used for image compression, capable of high compression ratios with its near-lossless feature. Due to data dependencies between sequentially processed samples, challenges arise when designing high throughput hardware accelerators for logic resource constrained systems. This work explores the use of the weight update scheme proposed by Jia et al. to increase the performance of the implementation by Vorhaug. Preliminary results using a high level model of the algorithm show a decrease in compression performance of *3.08%* over a range of absolute error limits from *0* to *15*. The proposed pipelined weight update stage is expected to increase the throughput of the design by a factor of three, while reducing the logic resource utilization.

AI Declaration

The generative AI tool ChatGPT has been used to assist with formatting of equations and tables. No such tools have been used to generate the text or figures of this document.

Contents

Abstract . . . . . **ii**
AI Declaration . . . . . **iii**
Contents . . . . . **iv**
1 Introduction **1**
2 Background **2**
2.1 Hyperspectral Image Compression 2
2.1.1 Image characteristics and sample ordering 3
2.1.2 Predictor overview 5
2.1.3 Local sums and differences 6
2.1.4 Weight vector and prediction calculation 7
2.1.5 Quantization and sample representatives 8
2.1.6 Weight updates 8
2.1.7 Data dependencies 9
2.1.8 Parameter selection 10
2.2 Evaluating Coding Performance 10
2.3 Hardware Accelerators 11
3 State-of-the-Art **13**
3.1 Overview 13
3.2 FID Ordering 15
3.3 Mathematical Optimization Approach 16
3.4 Reduced Weight Update Frequency 17
4 Previous Work **18**
4.1 High Level Model 18
4.2 Implementation by Vorhaug 18
5 Implementation and Verification **21**
5.1 Modifications to High Level Model 21
5.2 CNES Provided Software 21
5.3 Reduced Weight Updates 22
5.4 Verification 23
6 Results **25**
7 Discussion and Further Work **28**
7.1 Compression Performance 28
7.2 Hardware Implementation 28
7.3 Image Quality 29
7.4 Band-dependant Error Limits 29
7.5 Image Reconstruction 29
8 Conclusions **31**
Bibliography **32**

Chapter 1

Introduction

Hyperspectral imaging (HSI) is an important tool in many areas, including medical, food industry and earth observation [3]. With the advent of small satellite technology, the availability of space remote sensing, including HSI, has increased [4]. Smallsats, and hereunder cubesats, are often developed under limited budgets, using components off the shelf (COTS) to minimise cost and development time. This places a higher demand on the development of efficient hardware, sensors and algorithms for the acquisition and processing of data for such missions.

The HYPerspectral Smallsat for Ocean observation (HYPSO) is a project at the Norwegian University of Science and Technology (NTNU) focusing on hyperspectral imagery for ocean observation. The HYPSO-1 and HYPSO-2 satellites, launched in 2022 and 2024 respectively, both carry hyperspectral imagers capable of capturing data in the spectral range within wavelengths 400-800nm [5]. This spectral range allows monitoring of algal blooms, a phenomenon giving important clues as to the health of marine environments and ecosystems [6].

The sensors onboard the HYPSO satellites produce images with 120 spectral bands at spatial sizes of 598 lines and 1092 samples for HYPSO-2, and 684 by 956 for HYPSO-1 [5]. This, combined with the addition of meta data such as georeferencing and anomaly detection, produce large amounts of data to be stored, processed and downlinked. A typical raw HYPSO-1 image has a size of 160MB, and the, somewhat conservative, downlink capability during a single orbital revolution is 75MB. In order to achieve acceptable data latency it is evident that one must utilise image compression before downlinkning the data. Small satellites often operate at strict power budgets, and with limited hardware resources. With many onboard systems to manage, processor time is also limited. Commercially available Field Programmable Gate Arrays (FPGAs) tailored for space applications exist. By combining the flexibility of reconfiguration and speed of a custom hardware design, these are the ideal choice for implementation of HSI compression.

This work explores the possibility of using reduced weight update frequencies in the CCSDS-123.0-B-2 compression algorithm to increase the throughput of the implementation by Vorhaug [2]. Improvements in performance should not come at significant cost in terms of hardware resource utilization. The viability of the method is analyzed using a high level model of the algorithm before a proposed microarchitecture is presented.

The rest of this document is organized as follows. Section 2 gives an introduction to hyperspectral imagery and an overview of the relevant concepts of the CCSDS-123.0-B-2 compression standard. A review of the current state-of-the-art in the field is given in section 3, presenting the different implementation approaches that have been suggested and attempted. Sections 4 and 5 details the implementation for FPGA by Vorhaug before presenting the suggested modifications to allow a reduced weight update frequency. Sections 6 and 7 presents and discusses the results before conclusions are given in section 8.

Chapter 2

Background

This section introduces the use of hardware accelerators to improve system performance and quality metrics. Furthermore, it gives an overview of the CCSDS-123.0-B-2 compression algorithm, focusing on the adaptive weight updating scheme.

2.1 Hyperspectral Image Compression

The Consultative Committee for Space Data Systems (CCSDS) is a collaborative group working on the development of standards and algorithms for space applications. CCSDS Data Compression Working Group has released several algorithms for the compression of multi- and hyperspectral images at different stages during the compression process [7]. In 2012 the committee released the CCSDS-123.0-B-1 specification for lossless compression, hereby referred to as *issue 1*. This was in 2019 followed by the CCSDS-123.0-B-2, referred to as *issue 2*, introducing support for near-lossless compression. The following sections give a brief overview of the issue 2 algorithm, and its differences with issue 1. For a detailed walk-through see the CCSDS specification The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019 [8].

The issue 1 and issue 2 standards formalize the compression techniques presented in the works by Klimesh and Keymeulen et al. respectively [9–11]. Issue 1 is based on the Fast Lossless (FL) algorithm, encoding differences between actual sample values and estimated (predicted) sample values, known as prediction residuals. This is a common technique within image compression that exploits the statistical correlation between neighbouring image pixels to reduce the entropy in the data, allowing a more efficient encoding. The prediction method used is a form of discrete-time linear adaptive filtering with binary reinforcement like the method described in the work by Gersho in [12]. Estimates z_k of the current sample d_k are produced by observing samples x_k of a random process that are statistically dependant on the true sample values d_k. A linear preprocessor Φ produces a vector $\mathbf{u}_k$ from the observed samples as

$$\mathbf{u}_k = \Phi(x_k, x_{k-1}, \ldots). \tag{2.1}$$

The predicted value is then produced as

$$z_k = \mathbf{c}_k \cdot \mathbf{u}_k, \tag{2.2}$$

where $\mathbf{c}_k$ is a weight vector that is updated at each time instant. Corrections $\Delta\mathbf{u}_k$ are generated based the prediction residuals found by $z_k - d_k$. As the weight vector is updated the prediction residual is minimized, given a steady state in which sample values exhibit slow changes. After a sharp change in sample values the adaption will need some time to re-stabilize.

Results from the prediction stage are passed to an entropy encoder, where image data is encoded with predetermined, variable length code words. Common values in the data can then be encoded using fewer bits than less common values. For a discussion on the effectiveness of different encoding techniques see the work by Rahman and Hamada in [13].

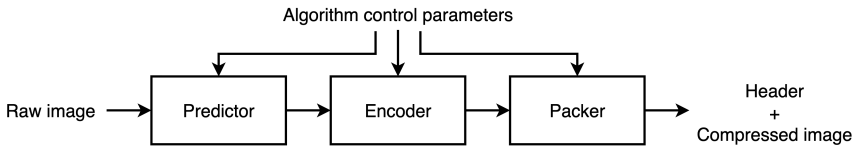

Figure 2.1: Block diagram of the issue 1 and issue 2 compression algorithms.

Figure 2.1 shows a block diagram of the issue 1 and issue 2 compression algorithms. The methods used are causal, meaning that the raw image samples are processed sequentially in a single pass, allowing fast implementations and minimal temporary storage. Samples are fed to the predictor stage, where sample values are predicted using the aforementioned adaptive linear prediction method, using a small neighborhood of spatially and spectrally adjacent pixels [14]. The results of the predictor are passed on to an encoder stage in which the image data is encoded into its compressed format. Each of the stages in the algorithm are controlled with a set of parameters. In the final stage these parameters are packed into a header which is shipped together with the compressed image as they are needed by the decoder in order to correctly reproduce the image.

Issue 2 is based on the FL Extended (FLEX) algorithm, which extends FL to support near-lossless compression. The notion of near-lossless, as compared to lossy compression, is that the user is provided with fine tuned control over the per sample error limit through setting an absolute and a relative limit. The standard also allows band-dependant error limits, allowing higher compression of less important spectral bands [7]. This is useful in the context of the HYPSO project, where the outer spectral bands tend to contain more noise [15].

Another important new feature of issue 2 is the addition of a hybrid-entropy encoder, in addition to the sample-adaptive and block-adaptive encoders of issue 1. The work done by Chatziantoniou et al. suggests an architecture, written in portable VHDL, for such a hybrid-entropy encoder achieving a throughput of 341 $Msamples/s$ [14]. The encoder implemented by Báscones et al. achieve similar speeds and outperforming that of Chatziantoniou et al. for the near-lossless mode [16]. These results show that high throughput implementations of the issue 2 hybrid-entropy encoder are possible, outperforming those of issue 1 for the near-lossless mode. For the remainder of this document the focus will be on the predictor stage of the compressor.

The notation used in the following sections is the same, unless otherwise specified, as those used by the CCSDS specification in [8]. Most notations will be mentioned in text, but not all are relevant for this discussion. For a full list, and the readers convenience, an overview is available in Table 2.1.

2.1.1 Image characteristics and sample ordering

Figure 2.2 shows the layout of a hyperspectral image, known as a data cube. Image dimensions are denoted as N_z, N_x and N_y, where z is the spectral band, and x and y the spatial dimensions. The dynamic range D is the bit depth of the image data. Samples of the same y-coordinate are referred to as frames. Within a frame, samples of the same band (z-coordinate) are denoted as lines. The image sensors onboard the HYPSO satellites use an image acquisition technique called pushbrooom. The pushbroom imager scans along the y-direction, capturing entire frames in sequential order [5]. Finally, samples of the same spatial coordinates x and y may be referred to as a pixel.

A sample in the three dimensional data cube can be referenced using its three coordinates as $S_{z,y,x}$. To simplify notation, the standard combines the spatial dimensions x and y in the spatial index t, given by

Table 2.1: Issue 2 predictor symbol table.

Symbol	Name
$s_z(t)$	Image sample
$\sigma_z(t)$	Local sum
$U_z(t)$	Local difference vector
$d_z(t)$	Central local difference
$\hat{d}_z(t)$	Predicted central local difference
$\check{s}_z(t)$	High-resolution predicted sample
$\tilde{s}_z(t)$	Double-resolution predicted sample
$\hat{s}_z(t)$	Predicted sample
$\Delta_z(t)$	Prediction residual
$m_z(t)$	Maximum error
$q_z(t)$	Signed quantizer index
$s_z^{'}(t)$	Clipped quantizer bin center
$s_z^{''}(t)$	Sample representative
$\theta_z(t)$	Predicted sample and sample endpoint difference
$\delta_z(t)$	Mapped quantizer index
$e_z(t)$	Double-resolution prediction error
$W_z(t)$	Weight vector

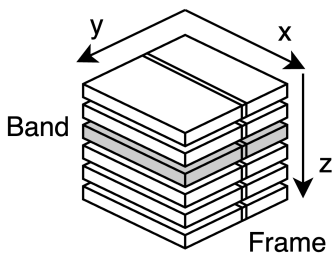

Figure 2.2: Dimensions of a hyperspectral image. Samples of the same z and y coordinates form bands and frames respectively. Samples in the same band of a frame are denoted as a line.

$$t = y \cdot N_x + x, \tag{2.3}$$

so that samples can be referred to as $s_z(t)$.

When feeding the samples to different stages of the algorithm it is important that the sample ordering is carefully considered. Two main categories of sample orderings exist, that are both supported by the standard: band sequential and band interleaved. For band sequential (BSQ) ordering, samples are processed one band at a time. Meaning that coordinates are iterated in the (x, y, z) order. Of band interleaved ordering, two special cases exist: band interleaved by pixel (BIP) and band interleaved by line (BIL), where coordinates are iterated in the (z, x, y) and (x, z, y) orders. The standard defines the interleaving depth as the number of line samples that interleaves each new band. This means that the interleaving depths of BIP and BIL ordering are 1 and N_x respectively. Figure 2.3 shows the BIL processing order, where image frames are processed sequentially.

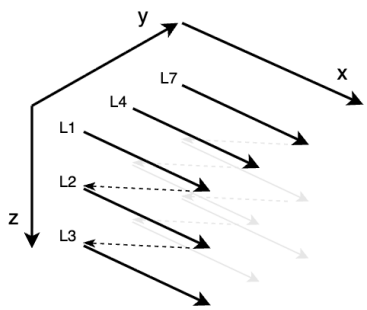

Figure 2.3: Band interleaved by line (BIL) processing order. Lines (L1 to L3) of the first frame are processed before the first line (L4) of the second frame.

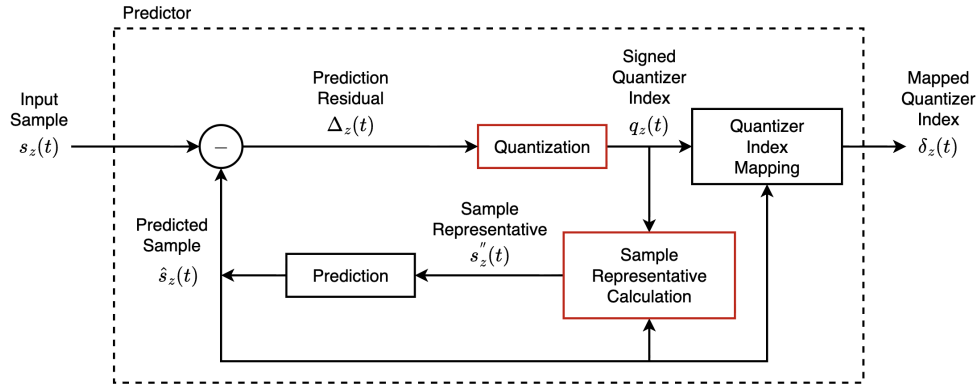

Figure 2.4: Block diagram of the issue 2 predictor, recreated from [7]. The new blocks as compared to issue 1 are marked in red. See Table 2.1 for symbol names.

2.1.2 Predictor overview

Figure 2.4 shows a block level view of the issue 2 predictor stage. To allow near-lossless compression, the issue 2 predictor introduces a quantization step, and sample representatives, marked in red. In the quantization step, the difference between input samples $s_z(t)$ and predicted sample values $\hat{s}_z(t)$, named prediction residuals $\Delta_z(t)$, are grouped into bins. Thus collecting nearby values, reducing the possible sample value ranges and allowing higher compression ratios. The signed quantizer indices $q_z(t)$ are mapped to unsigned indices, the mapped quantizer indices $\delta_z(t)$, which are passed on to the encoder. The size of the quantizer bins, and thus the loss in information, is controlled by the maximum error $m_z(t)$ given by

$$m_z(t) = \min\left(a_z, \left\lfloor \frac{r_z|\hat{s}_z(t)|}{2^D} \right\rfloor\right), \tag{2.4}$$

where r_z and a_z are the peak relative and peak absolute errors (PAE) respectively. If only absolute error limits are used, $m_z(t) = a_z$, and if only relative limits are used then $m_z(t) = \left\lfloor \frac{r_z|\hat{s}_z(t)|}{2^D} \right\rfloor$. As mentioned, the standard allows error limits to be specified on a per band basis, meaning that vectors of size N_z containing the absolute and relative error limits must be provided.

Because the real sample values are not available to the decoder, the predictor uses sample representatives $s''_z(t)$ during sample prediction. The use of sample representatives is one of the main reasons for the challenges of implementing the algorithm for high throughput hardware applications [17], as neighbouring sample representatives must be calculated before the processing of the current sample can commence.

The decoder reconstructs the quantizer indices $\delta_z(t)$ from the compressed bitstream. Samples values are then reconstructed casually, allowing calculation of the predicted sample values $\hat{s}_z(t)$ by use of previously decoded samples [11]. This process requires that the same prediction model of the encoder can be reproduced, prompting the need for the mentioned sample representatives. The reconstructed predicted sample values, in combination with the error limits, are then used to calculate the decompressed samples values.

2.1.3 Local sums and differences

Figure 2.5 shows a typical prediction neighborhood, where the sample representatives $s''_z(t)$ of neighbouring samples are used to calculate the predicted sample value $\hat{s}_z(t)$. The user configurable parameter P determines the number of previous spectral bands to be used during prediction calculation.

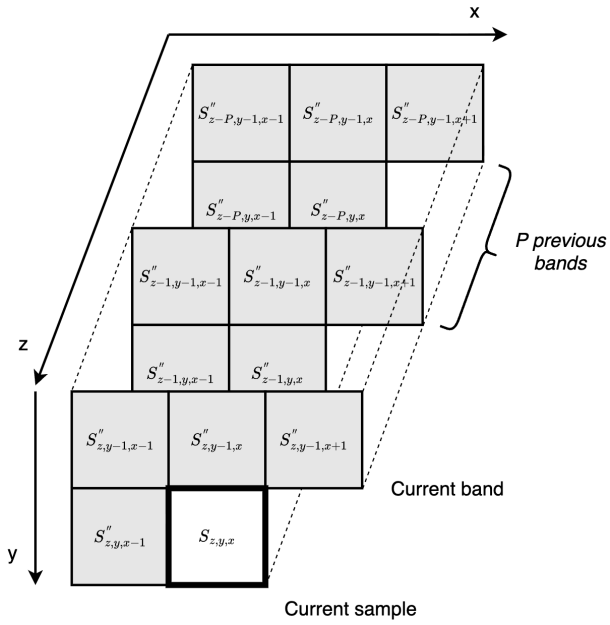

Figure 2.5: Typical prediction neighborhood. Spatially adjacent samples in the current band are used to find the local sum of the current sample. This combined with the local sums of P previous bands are used to find the predicted value. Figure recreated from [8].

The first two stages of prediction are the calculations of local sums and local differences. For each sample we find the local sum $\sigma_z(t)$ by adding up sample representative values of spatially neighbouring samples. Which samples, and their weighting, is determined by the selected prediction neighborhood. Here we distinguish between neighbor- and column-oriented local sums, both of which can be configured as wide or narrow.

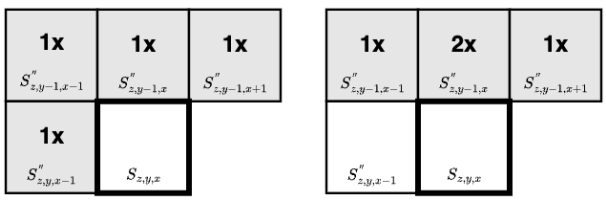

Figure 2.6: The wide (left) and narrow (right) neighbor-oriented configurations for local sum calculations. Dependencies on the previous sample in the line $(x-1)$ are avoided in the narrow variant.

Figure 2.6 shows the wide and narrow configurations for neighbor-oriented local sums, with their respective weights. The key observation is that narrow local sums remove the dependence on the sample representative of the previous sample in the x-direction, $x-1$. The details of the column-oriented configuration and of the corner cases are not important for this work, but, as an example, the full definition of the narrow neighbor-oriented local sum configuration is given by

$$\sigma_{z,y,x} = \begin{cases} s''_{z,y-1,x-1} + 2s''_{z,y-1,x} + s''_{z,y-1,x+1}, & y > 0,\ 0 < x < N_X - 1, \\ 4s''_{z-1,y,x-1}, & y = 0,\ x > 0,\ z > 0, \\ 2\left(s''_{z,y-1,x} + s''_{z,y-1,x+1}\right), & y > 0,\ x = 0, \\ 2\left(s''_{z,y-1,x-1} + s''_{z,y-1,x}\right), & y > 0,\ x = N_X - 1, \\ 4s_{\text{mid}}, & y = 0,\ x = 0,\ z = 0. \end{cases} \tag{2.5}$$

Local sums are used to find the central local difference $d_z(t)$. This quantity is, as we will see, needed later by the prediction calculation and weight update stages. The central local differences of the samples in the prediction bands at the same spatial position t as the current sample form the local difference vector $\mathbf{U}_z(t)$, given as

$$\mathbf{U}_z(t) = \begin{bmatrix} d_{z-1}(t) \\ d_{z-2}(t) \\ \vdots \\ d_{z-P_z^*}(t) \end{bmatrix}, \tag{2.6}$$

where $d_{z-i}(t)$ denotes the element corresponding to the ith prediction band. i takes values in the range $0 < i \leq P_z^*$ where P_z^* is the number of used prediction bands, given by

$$P_z^* = \min(z, P), \tag{2.7}$$

and each element $d_z(t)$ as

$$d_z(t) = 4s''_z(t) - \sigma_z(t). \tag{2.8}$$

The standard defines two configurations for local differences, full and reduced prediction mode, where the definition provided in Equation 2.6 is of the reduced mode. Full prediction mode extends the definition by adding local differences corresponding to three extra spatially neighbouring samples. Its exact definition can be found in the standard. This adds additional data dependencies that can be avoided by selecting the reduced prediction mode.

The mechanisms described in this section form the preprocessing step in Equation 2.1 of the linear adaptive filter, where the resulting vector $\mathbf{u}_k$ is the central local difference vector $\mathbf{U}_z(t)$.

2.1.4 Weight vector and prediction calculation

As in Equation 2.2 the central local difference vector $\mathbf{U}_z(t)$ is used to find the predicted central local difference $\hat{d}_z(t)$ as

$$\hat{d}_z(t) = \mathbf{W}_z(t) \cdot \mathbf{U}_z(t). \tag{2.9}$$

where, for the reduced prediction mode, the weight vector $\mathbf{W_z(t)}$ is given by

$$\mathbf{W}_z(t) = \begin{bmatrix} \omega_z^{(1)}(t) \\ \omega_z^{(2)}(t) \\ \vdots \\ \omega_z^{(P_z^*)}(t) \end{bmatrix}, \tag{2.10}$$

with $\omega_z^{(i)}(t)$ denoting the element corresponding to the ith prediction band. The standard allows two methods of weight initialization: standard and custom. For standard weight initialization the weights are initialized as

$$\omega_z^{(1)}(1) = \frac{7}{8}2^{\Omega}, \qquad \omega_z^{(i)}(1) = \left\lfloor \frac{1}{8}\,\omega_z^{(i-1)}(1) \right\rfloor, \quad i = 2, 3, \ldots, P_z^*, \tag{2.11}$$

where Ω is the weight bit resolution. Weights are presented as signed integers using $\Omega + 3$ bits. This gives minimum and maximum weight values of

$$\omega_{\min} = -2^{\Omega+2},\ \omega_{\max} = -2^{\Omega+2} - 1. \tag{2.12}$$

Custom weight initialization allows the weights to be set to an arbitrary (within the bit resolution) value which may be chosen based on image or sensor characteristics.

Predicted central local differences $\hat{d}_z(t)$ are used to produce the final predicted sample value $\hat{s}_z(t)$ by first calculating the high-resolution predicted sample value $\breve{s}_z(t)$ and the double-resolution predicted sample value $\tilde{s}_z(t)$, whose definitions will be left out of this text.

2.1.5 Quantization and sample representatives

The difference between the predicted samples value and the real sample produce the prediction residual $\Delta_z(t) = s_z(t) - \hat{s}_z(t)$. This is passed to the quantization step, producing the aforementioned signed quantizer indices $q_z(t)$. Finally, the predictor outputs, the mapped quantizer indices $\delta_z(t)$, and sample representatives $s_z^{''}(t)$ are calculated.

2.1.6 Weight updates

For each time increment (each new sample) the weight vector is updated using the sign algorithm, also known as the binary reinforcement algorithm [11, 12]. As described, the algorithm uses the difference between predicted and actual sample values to produce a weight update, or offset. The simplicity of the sign algorithm is that it only uses the sign of this difference to decide whether the weights should be increased or decreased. This reduces the complexity of the algorithm at the cost of convergence time towards a steady state.

Using the notation presented in [1], the weight offset is given by

$$\omega_{z,\text{offset}}^{(i)}(t) = \left\lfloor \frac{1}{2}\left(2^{-\rho(t)} \cdot d_{z-i}(t) \cdot \operatorname{sgn}^{+}[e_z(t)] + 1\right) \right\rfloor, \tag{2.13}$$

where $e_z(t)$ is the double-resolution prediction error given by

$$e_z(t) = 2s_z'(t) - \tilde{s}_z(t). \tag{2.14}$$

Because of the near-lossless feature of the algorithm, the clipped quantizer bin center $s_z^{'}(t)$ is used instead of the real sample value to find the prediction residual for weight updates. If error limits are set to zero, meaning lossless compression is used, $s_z^{'}(t) = s_z(t)$. Furthermore, the weight update scaling exponent $\rho(t)$ is given by

$$\rho(t) = \operatorname{clip}\left(\nu_{\min} + \left\lfloor \frac{t - N_x}{t_{\text{inc}}} \right\rfloor, \{\nu_{\min}, \nu_{\max}\} \right) + D - \Omega, \tag{2.15}$$

where ν_{min}, ν_{max} and t_{inc} are user configurable parameters that tune how fast the weights adapt to changes in the image.

Finally, the weight offset is added to the weight of the previous sample, giving the updated weight as

$$\omega_z^{(i)}(t+1) = \text{clip}\left(\omega_z^{(i)}(t) + \omega_{z,\text{offset}}^{(i)}(t), \{\omega_{min}, \omega_{max}\}\right), \tag{2.16}$$

where the clip function clips the result between the minimum and maximum weight values ω_{min} and ω_{max} (Equation 2.12).

2.1.7 Data dependencies

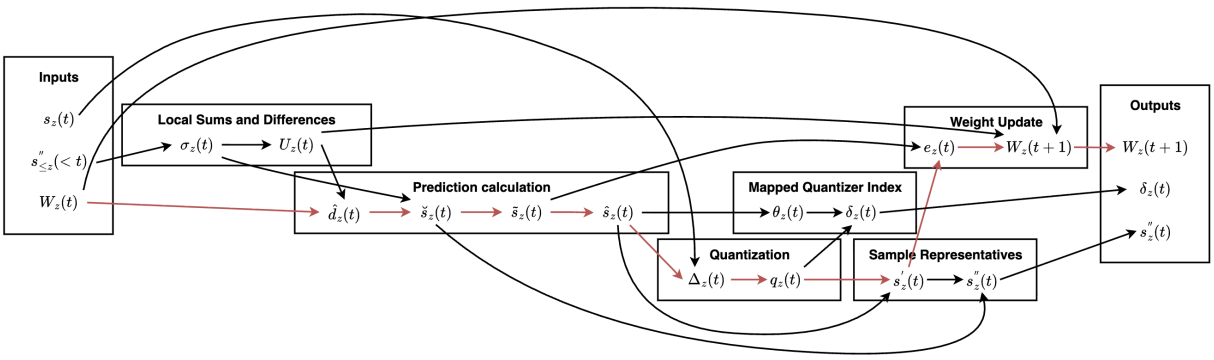

Figure 2.7: Data flow of the issue 2 predictor stage, recreated from [17]. See Table 2.1 for symbol names.

Figure 2.7 shows the data flow of the issue 2 predictor. We distinguish between spectral, $z-1$ dependencies, and spatial, $t-1$ dependencies. An example of a $t-1$ dependency is the weight update stage, in which the weights from sample $t-1$ are required to calculate the updated weight for sample t. The central local difference vector $\mathbf{U}_z(t)$ is an example of a $z-1$ dependency. Here the central local differences of the prediction bands are required. These dependencies create a challenge for pipelined hardware implementations as they must be resolved before the processing of the next sample can commence.

By carefully selecting the algorithm configuration and input sample ordering, some of these challenges may be resolved, as will be covered later in this document. This leaves the weight update calculation, whose dependencies are marked with red arrows in the figure, as the critical path.

Table 2.2: The number of processed samples interleaving $z-1$ and $t-1$ dependencies.

	BSQ	**BIP**	**BIL**
$z-1$	$N_x \cdot N_y$	1	N_x
$t-1$	1	N_z	1

Assuming one new sample is processed every clock cycle, the number of cycles available for resolution of the data dependencies are determined by the sample ordering. Table 2.2 gives an overview of the number samples interleaving $z-1$ and $t-1$ dependencies for each of the three main orderings BSQ, BIP and BIL. Their derivation is as follows:

1. BSQ ordering iterates samples in the (x, y, z) order. Entire bands are processed before starting on the next one, meaning $N_x \cdot N_y$ samples interleave each band. Interband pixels are processed sequentially, leaving only a single cycle for $t-1$ dependencies.
2. BIP ordering iterates samples in the (z, x, y) order. All samples of the same spatial index t are processed before proceeding to $t+1$. This leaves $z-1$ dependencies a single cycle to resolve, and N_z for $t-1$ dependencies.
3. BIL ordering iterates samples in the (x, z, y) order. Samples in a line are processed sequentially before starting the next band, giving N_x samples for $z-1$ dependencies and a single cycle for $t-1$.

2.1.8 Parameter selection

As briefly mentioned, a range of user configurable parameters are available for tuning the operation of the algorithm. One such example are the v_{min} and v_{max} parameters that control the adaption rate of the predictor. The work by Blanes et al. presents a thorough analysis of the affects of tuning these parameters, suggesting a set of values that are well suited for most image characteristics [18]. These values will be used as the basis of the parameters selected for this work.

When selecting the error limits one must balance the compression performance and the introduced compression noise. The latter involves minimizing quality metrics, as will be detailed in the next section, such as mean squared error and signal-to-noise ratio. For images containing sample areas of varying dynamic range, using only absolute error limits can cause unwanted effects. Coastal images, typical for Hypso images, contain dark regions of water. These regions have low dynamic range, meaning that the absolute error limit may set these entire areas to the same value. For images dark water regions are part of the main image objective, this effect is not acceptable. By use of relative error limits, the dynamic range can be preserved across the image. A detailed analysis of this phenomenon can be found in [11], concluding that to achieve high image quality a combination of absolute and relative error limits should be used.

2.2 Evaluating Coding Performance

Image data contains inherent noise from the sensor, and in the case of HSIs from the HYPSO mission, atmospheric noise from light reflecting in the atmosphere [15]. In a lossless compression scheme the original image can be reconstructed perfectly. With the near-lossless feature of issue 2 additional noise is introduced. We must balance the error limits to achieve a high compression ratio while not loosing too much information. From [1] the compression ratio (CR) of the encoded image is given by

$$\text{CR} = \frac{N_x \cdot N_y \cdot N_z \cdot D}{\text{compressed data size (bits)}}. \tag{2.17}$$

Peak signal to noise ratio (PSNR) is a common metric used to evaluate the image noise introduced by compression [19]. For a single spectral band z the PSNR is given by

$$\text{PSNR}_z = 10 \cdot \log_{10}\left(\frac{\text{MAX}_I^2}{\text{MSE}_\text{z}}\right), \tag{2.18}$$

where the mean square error MSE is given by

$$\text{MSE}_\text{z} = \frac{1}{N_x N_y} \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \left[I_z(i,j) - K_z(i,j)\right]^2. \tag{2.19}$$

Here I_z and K_z are the spectral band z of the original and decoded image respectively. MAX_I denotes the maximum possible value of the data in the original image. To find the PSNR of the entire image cube we take the average of the per band PSNRs as

$$\text{PSNR}_{HSI} = \frac{1}{N_z} \sum_{z=0}^{N_z-1} \text{PSNR}_z. \tag{2.20}$$

2.3 Hardware Accelerators

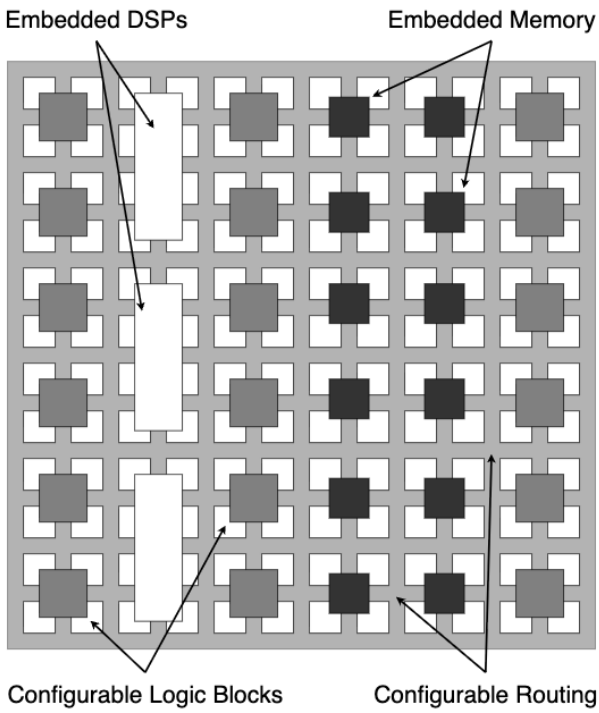

Figure 2.8: A typical FPGA architecture. Block memories, digital processing units and configurable logic blocks are embedded into a routing matrix, providing high flexibility. Figure recreated from [20].

Embedded real-time systems often operate under strict deadlines. Missing these deadlines can be detrimental to system performance and correct functioning. At the same time, designers must balance budgets on power, area and cost. An effective approach to designing robust and responsive systems is to utilize multiprocessors [21]. Multiprocessor systems contain multiple processing elements (PE), that can be optimized for specific functions, performing tasks in parallel. It is well known that the use of simpler units running in parallel can provide the same performance at a lower power and cost [21, 22].

In addition to programmable PEs, e.g. central processing units (CPU), which provide great flexibility, multiprocessor systems can achieve enormous performance improvements by the use of specialized hardware accelerators [21]. One such example is the graphical processing unit (GPU), which has become a staple of modern personal computers. Field-programmable gate arrays (FPGA) allow designers to design custom logic at low costs, as compared to the higher performance and costly application specific integrated circuits (ASIC). Most FPGAs also come at the added benefit of being reconfigurable, which provides flexibility for remotely deployed systems.

Figure 2.8 shows the layout of a typical FPGA. By use of look-up tables (LUT) and flip-flops (FF), configurable logic blocks (CLB) can be configured to provide any logic function. Units are mounted

in a large routing matrix, allowing custom routing between the CLBs. These two mechanisms provide the core functionality of the FPGA, and allow the implementation of any logic design. In order to increase efficiency in area, power and performance, it is common to add additional, specialized hardware units. Digital processing units (DSP) provide common arithmetic functions, while embedded block random access memory (BRAM) provides efficient data storage. The hardware resource utilization of a design for FPGA is typically measured in terms of the amount of LUTs, FFs, DSPs, and BRAM used.

Another way of utilizing parallelization to increase performance is pipelining. By separating data independent stages of the calculation with registers, known as pipeline registers, the clock frequency can be increased while still issuing new data for processing every cycle. This approach is widely used in hardware designs, for example in CPUs. The clock frequency is limited by the longest path between any two pipeline registers. For implementations of the issue 2 algorithm, the aforementioned data dependencies create problems when trying to achieve high degrees of pipelining.

Compared to software design, hardware design comes at increased complexity in terms of design and development. A typical design starts at a microarchitectural level before the design is converted to the register transfer level (RTL) using languages like VHDL, Verilog and Systemverilog. RTL code is then synthesized to a gatelevel netlist which, through a process called place and route (PnR), is mapped to a specific FPGA or for ASIC. To ensure correct functioning of the final result, extensive verification must be performed at all steps in the design process.

To alleviate the challenges in long design time of RTL code, a new design flow has emerged named high-level synthesis (HLS). The advantage of using the HLS approach is reduced development time, as the algorithm can be described in a high-level language like C or C++, which is then converted to RTL [23]. However, the tools for such a conversion create suboptimal results when compared to writing the implementation directly in RTL using languages like VHDL or Systemverilog, giving the designer fine-grained control over optimizations.

The satellites of the HYPSO project use multiprocessor system-on-chips (MPSoC) from AMD (formerly Xilinx), namely the Zynq chips [5]. These COTS embed two ARM processing units, a simple GPU, and several useful peripherals including analog-to-digital converters (ADC), timers and support for various communication protocols. The system runs the Linux operating system, providing tools for management of complex tasks and threads. Additionally, the chips include embedded programmable logic (PL), equivalent to an embedded FPGA. Embedding the PL directly in the MPSoC allows custom logic to directly access the main data buss and other systems on the chip, including direct memory access (DMA) for fast and CPU independent memory accesses. These features give great benefits in terms of efficiency in speed and power as compared to separate FPGAs.

Chapter 3

State-of-the-Art

This section reviews the current state-of-the-art of hardware implementations of the CCSDS-123.0-B-2 compression algorithm. An overview is given of the different proposed solutions and their viability for deployment in a project like HYPSO.

3.1 Overview

Several implementations of issue 1 exist, achieving high throughputs that reach the real-time requirements of hyperspectral image sensors, the highest of which reaching $1387 Msamples/s$ [24]. The parallel FPGA implementation by Orlandic et al. is in active use onboard the HYPSO satellites, with a throughput of $750 Msamples/s$ [5, 25]. For HYPSO-1 images the compression ratio using issue 1 is about 50% reduction in image size, or a CR of 1.5 (Equation 2.17) [2]. Issue 2 is capable of significantly higher compression ratios, depending on the error limit, motivating a switch over from issue 1.

The data dependencies of issue 2 introduced by the inclusion of sample representatives creates significant challenges for hardware implementation. The image sample ordering, introduced in section 2.1.1, impacts the influence of the dependencies. For the three main orderings, BIP, BSQ and BIL, pixels are traversed in the coordinate orders (z, x, y), (x, y, z) and (x, z, y) respectively. Choosing the BIP ordering results in a spectral dependency in the calculation of the local differences, which requires the local sum of the previous sample in the same band. By using the BSQ or BIL orderings, spectrally neighbouring samples are placed far enough apart in the processing order that this dependency has time to resolve. This, however, introduces a spatial dependency related to the weight updating scheme, which requires the sample representatives of spatially neighbouring samples (red path in Figure 2.7). Three main approaches have been proposed to alleviate these dependency problems, which will be discussed in the following sections.

The work by Barrios et al. introduces the first implementation for FPGA of issue 2 in open literature [23]. This initial attempt followed a High-Level Synthesis (HLS) methodology, reaching a somewhat underwhelming throughput of 17.86 $Msamples/s$. The design does not make any special considerations as to the data dependencies of the algorithm, resulting in an almost fully serial processing of the samples and thus an unacceptably low throughput. The authors suggest using multiple compression units in parallel to increase total system performance. This is made possible by segmenting the image before compression, an approach successfully utilized by implementations of issue 1 [24].

Báscones et al. presents the first viable real-time implementation. By introducing a novel sample ordering scheme, named frame interleaved by diagonal (FID), samples are placed far enough apart that the mentioned dependencies are avoided. This allows a deeply pipelined design, reaching a throughput of $285 Msamples/s$ [16] using VHDL RTL. The new sample ordering requires the use of internal reordering buffers at the beginning and end of the processing pipeline to convert from the

conventional BIP ordering, adding complexity and logic resource usage.

Chatziantoniou et al. identifies that the in-loop quantization step (Figure 2.4) can be extracted and performed prior to the prediction stage [26]. This removes the need for sample representatives, effectively leaving a predictor similar to that of issue 1, void of the challenging dependencies. The single core throughput of this approach matches that of Báscones et al., and by use of segmentation and five cores they reach $1375 Msamples/s$. However, pre-quantization comes at the cost of a reduction in compression ratio as compared to that of the inline quantizer of the standard.

By using the FID ordering or pre-quantization in combination with image segmentation, high throughputs can be achieved. For systems with constraints on hardware logic resources, like the HYPSO mission, these approaches are not viable. Therefore, a different line of optimization must be pursued.

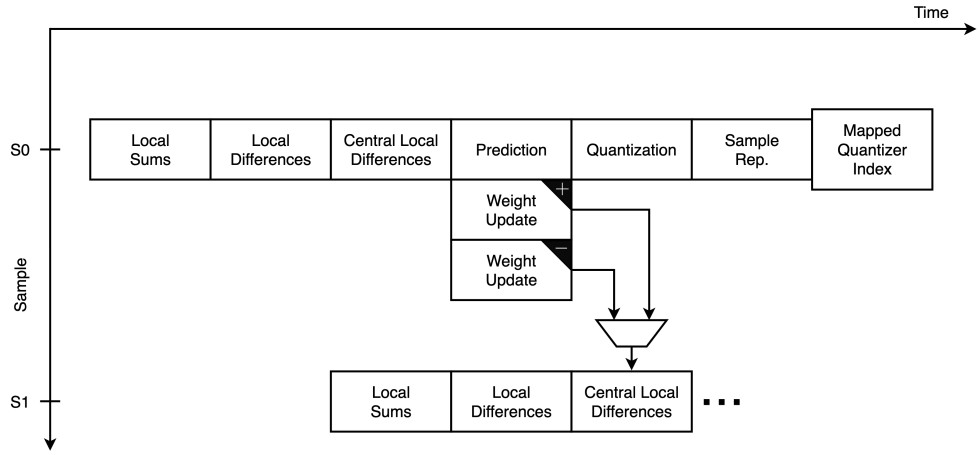

Figure 3.1: Suggested pipeline stages for hardware implementation using the mathematical optimization proposed by Sánchez et al. Recreated from [17].

Sánchez et al. identifies that by using a combination of the predictor configurations narrow local sums and reduced prediction mode, with the BIL ordering, most spatial dependencies on sample representatives are removed [17]. Furthermore, they suggest a mathematically equivalent way of calculating the first step of the weight update, namely the double-resolution prediction error $e_z(t)$, removing its dependency on the quantization and sample representative steps (Figure 2.7). This, in combination with a speculative calculation of the possible outcomes, allows the weight update stage to be calculated in parallel with the prediction stage. As seen in the suggested pipeline shown in Figure 3.1, this allows a significantly higher degree of pipelining of the design. This approach has been successfully implemented by Vorhaug using VHDL RTL, achieving a throughput of $110 Msamples/s$ with a hardware logic utilization of roughly 1/3rd of that of the core proposed by Báscones et al. in terms of LUTs, and 1/8th in terms of FFs [2, 16]. This is the issue 2 compliant implementation with the lowest hardware footprint in open literature, and has been tested onboard the HYPSO-1 satellite, achieving a reduction in image size of 66% in the lossless compression mode.

By carefully tuning the user configurable parameters related to weight updates, Barrios et al. suggest that weight updates can be skipped entirely, allowing full pipelining [27]. To avoid significant degradation in compression performance, careful consideration must be taken when selecting the initial weight vector for such a design. These initial values depend heavily on the image sensor and image objective, and thus the compressor is not able to run fully independently. Using the HLS methodology they reach a throughput of $235.4 Msamples/s$ with a somewhat lower hardware footprint than that of Báscones et al.

The mathematical optimization approach suggested by Sánchez et al. leaves the combined weight updating and prediction calculation stage as the critical path of the pipelined design (Figure 3.1),

constraining the obtainable maximum throughput [1, 17]. To alleviate this, Jia et al. suggest selectively skipping the weight updates [1]. This gives the weight update stage more time to complete, allowing a higher clock frequency at the cost of a slight degradation in compression performance. To combat this, the authors suggest a running correction to the applied weight updates which take the previous sample into consideration while keeping the new timing margins for the updates. Figure 3.3 shows a weight updating scheme with reduced frequency to every four samples. Using VHDL RTL their implementation of the issue 2 predictor reaches a throughput of 348.4$Msamples/s$, thus outperforming all the predictors discussed so far. The approach is also able to keep a small hardware footprint of only 3995 LUTs and 5050 FFs. For projects like HYPSO, the reduced weight updating frequency seems like a promising direction for the implementation of HSI compression.

Table 3.1: Throughputs of hardware implementations of the issue 2 algorithm in open literature.

Implementation	Throughput
Barrios et al. [23]	18 Msamples/s
Sánchez et al. [17, 27], predictor	231 Msamples/s
Barrios et al. [27], predictor	236 Msamples/s
Báscones et al. [16]	285 Msamples/s
Chatziantoniou et al. [26] 1 core	285 Msamples/s
Chatziantoniou et al. [26] 5 core	1375 Msamples/s
Vorhaug et al. [2]	110 Msamples/s
Jia et al. [1], predictor	348 Msamples/s

Table 3.1 gives an overview of the performance for the mentioned hardware implementations in terms of mega samples per second. Take note of the roughly three times improvement introduced by the reduced weight updates between the implementations of Vorhaug and Jia et al.

3.2 FID Ordering

The novel sample ordering proposed by Báscones et al. gives all data dependencies up to N_z cycles to resolve. This allows deep pipelining in all configuration of the standard, including the mentioned full prediction modes. To convert from, and back to, the conventional BIP ordering, reordering buffers of size $N_z \cdot N_z$ are added at the start and end of the predictor stage. This adds additional storage requirements of $2 \cdot N_z^2 \cdot D$ bits. Supporting all configuration also requires additional hardware resources, meaning that the increase in area is significant as compared to the mathematical optimization approach.

Table 3.2: The number of processed samples interleaving $z-1$ and $t-1$ dependencies, including the novel FID ordering in addition to the standard orderings.

	BSQ	BIP	BIL	FID
$z-1$	$N_x \cdot N_y$	1	N_x	N_z-1
$t-1$	1	N_z	1	N_z

Figure 3.2 shows how samples are processed in the FID ordering, with processed samples marked in white. Image frames are concatenated along the z-axis. Samples are then processed along a diagonal like presented in the figure. The number of samples interleaving dependencies can be found by counting along this diagonal, giving N_z-1 and N_z for $z-1$ and $t-1$ dependencies respectively. Table 3.2 gives an overview of the interleaving samples for the standard orderings with the inclusion of the FID ordering.

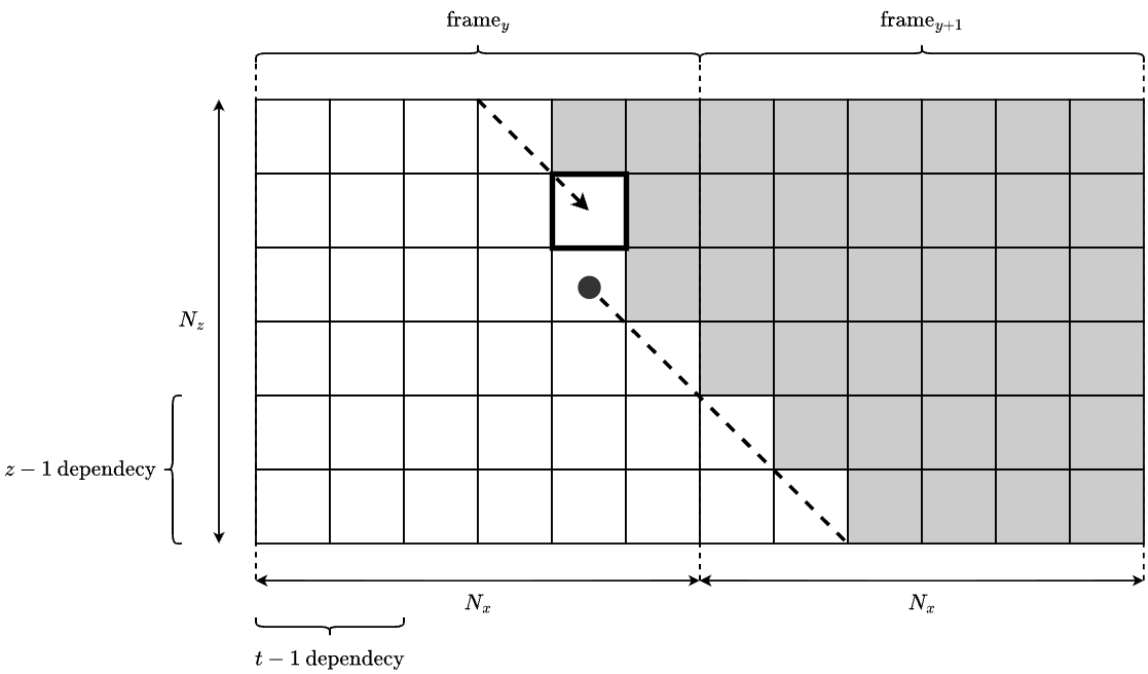

Figure 3.2: FID sample ordering with processed samples marked in white. Frames are concatenated along the z dimension, and samples are processed along the marked diagonal. This gives data dependencies time to resolve, allowing deep pipelining.

3.3 Mathematical Optimization Approach

The mathematical optimization of the sign calculation of the double-resolution prediction error presented by Sánchez et al. is defined as

$$\operatorname{sgn}^+\big[e_z(t)\big] = \begin{cases} -1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is odd,} \\ 1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is even,} \\ \operatorname{sgn}^+\big[s_z(t) - \hat{s}_z(t)\big], & \text{otherwise.} \end{cases} \tag{3.1}$$

From Equation 2.14 we can see that the calculation requires results from the sample representative step (Figure 2.7) which further depends on the results of the quantization step. The alternative, and equivalent, definition given in Equation 3.1 removes both of these dependencies, meaning that the sign calculation can be performed in parallel with the prediction calculation step as shown in the suggested pipeline in Figure 3.1.

As mentioned, the weight update stage forms a $t-1$ dependency, meaning that its calculation must be completed during a single clock cycle. This means that it cannot be divided into several steps for pipelining, resulting in the critical path of pipelined hardware implementations.

In addition to the equivalent formulation of the sign operation, the calculation path can be further reduced by a speculative calculation. Both of the two possible weight increments, corresponding to each of the sign values, are computed in parallel to the sign. Once all three calculations are complete the correct one is chosen.

3.4 Reduced Weight Update Frequency

The reduction in weight update frequency proposed by Jia et al. allows weight calculation to begin four cycles earlier. This means that the previously indivisible calculation can be split into four pipeline stages, drastically reducing the critical path for hardware implementations.

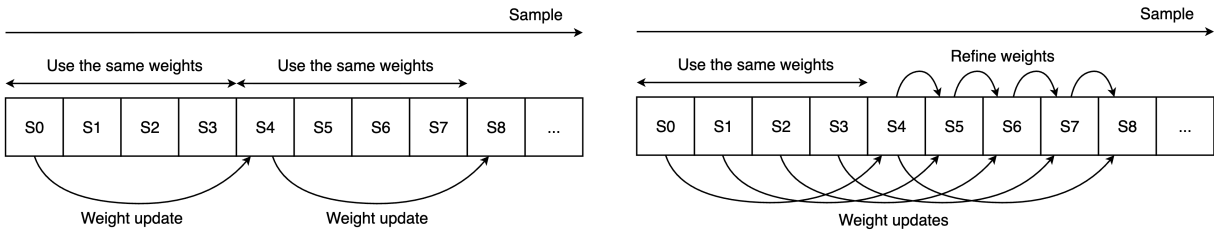

Figure 3.3: Reduced weight updating frequency allows a shorter critical path in hardware implementations, yielding higher throughput. Weight refinements take the previous weight into account, mitigating the loss in coding performance. Figures recreated from [1].

Figure 3.3 shows a graphical representation of the proposed solution. The left figure shows a scheme where weight updates are applied for every fourth sample in the processing order of a line. This comes at the cost of a reduction in compression performance, as the sign algorithm becomes less responsive to changes in the image. To alleviate this the authors suggest starting a new weight update calculation for every sample, which is applied four steps later showed in the right figure. Further, a refinement step is applied which takes the weight from the previous sample into account. The formal definition of this scheme is as follows.

$$\omega^{(i)}_{z,\text{unclipped}}(t+1) = \begin{cases} \omega^{(i)}_{z}(t), & x \leq 2, \\ \omega^{(i)}_{z}(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3), & x = 3, \\ \left\lfloor \frac{1}{2}\left(\omega^{(i)}_{z}(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3) + \omega^{(i)}_{z}(t)\right) \right\rfloor, & \text{otherwise.} \end{cases} \tag{3.2}$$

For the first three samples of each new line, the same weight is used. In the very first line this means using the initial weight value. Consecutive lines will here use the last weight values of the previous line. For the fourth sample the calculated weight using the first line sample is used, which at this point has had time to resolve. In the rest of the samples the refinement step is additionally applied, requiring only a single addition and one bit right shift. The expected gain in performance should be about a three to four times increase in clock frequency, assuming that no other steps in the calculation caps the performance.

Chapter 4

Previous Work

This work build upon that by Vorhaug in [2]. Before the proposed changes are presented in chapter 5, the relevant details of the high level model and compressor implementation are presented in this section.

4.1 High Level Model

For use as a golden reference during verification of the hardware implementation, Vorhaug et al. has implemented a high level model of issue 2 in [28]. The model is written in the programming language Python, providing easily modifiable code at an abstraction closely resembling the algorithmic description. The program stores intermediary values of the computation, like the weight values. This allows verification of intermediary values in the hardware simulations. The model is not optimized for speed or memory usage, resulting in slow precessing and large amounts of allocated memory. Tests using full Hypso images is therefore impractical during development.

4.2 Implementation by Vorhaug

The implementation by Vorhaug uses the mathematical optimization approach of Sánchez et al. for weight calculations. Additionally, the reduced prediction mode and narrows local sums are used to alleviate $t-1$ dependencies. Finally, the BIL sample ordering removes all $z-1$ dependencies, by interleaving them with N_x samples (Table 3.2).

Figure 4.1 shows a block level view of the implementation. Samples are fed to, and read from, the compressor using AXI handshakes, a simple protocol for on-chip data transmission developed by ARM. An optional reorder buffer transforms the order of incoming samples from BIP to BIL. This is to allow easy integration on Hypso-1, where samples are stored in BIP ordering. The verification framework uses the aforementioned high level python model as a golden reference. Intermediary values are read during simulation and compared to the results of the high level model. Table 4.1 summarizes the hardware resource utilization of the compressor as integrated on the MPSoC onboard the Hypso-1 satellite.

The predictor of the compressor uses a pipeline closely resembling that of Figure 3.1, with the indivisible weight update stage limiting the maximum clock frequency. Figure 4.2 shows the weight update module of the implementation. This block is duplicated for each of the prediction bands P. The $Weight^+$ and $Weight^-$ blocks speculatively compute the weight update as per Equation 2.16. In parallel, the sign of the double-resolution prediction error $e_z(t)$ is computed, allowing selection of the correct weight in the first multiplexer (MUX). The second MUX selects the initial weight value for the first sample of the band at $t=0$. For interband weight updates the weight is stored in a register for retrieval in the next cycle. Because samples are processed in the BIL ordering (Figure 2.3) each

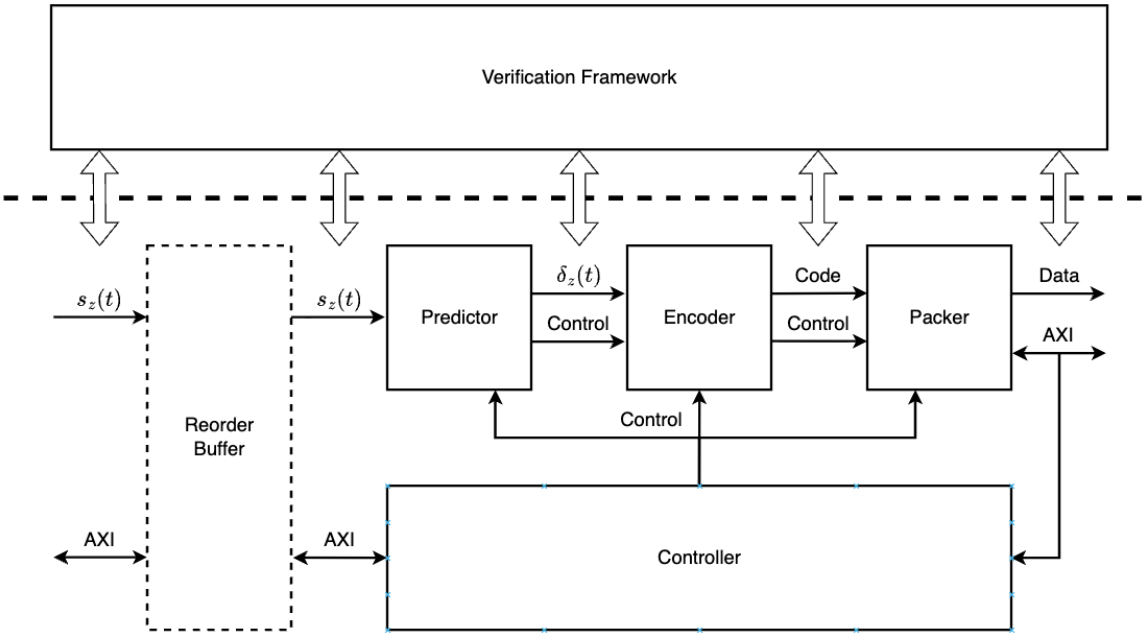

Figure 4.1: Hardware implementation by Vorhaug. An optional reorder buffer transforms sample order from BIP to BIL for use onboard Hypso-1 satellite. During simulation, the verification framework applies inputs and read intermediary and final outputs. These are compared with a golden model, written in python.

Table 4.1: Hardware resource utilization of the Vorhaug compressor as integrated on the Hypso-1 satellite.

	LUT (78600)	**FF (157200)**	**DSP (400)**	**BRAM (256)**	**Freq (Mhz)**
Issue 2 (Reorder)	5519 (7.0%)	2867 (1.8%)	4 (1.0%)	163.5 (61.7)%	50

frame is processed sequentially. This mean that the weight of the last sample of a line must be stored while the rest of the lines in the frame are processed. A first in first out (FIFO) buffer of size N_z is used for this purpose. At the end of the line, $x = N_x - 1$, the weight is pushed to the FIFO. At the first sample, $x = 0$, of the corresponding line of the next frame the weight has reached the head of the FIFO and is retrieved.

In Equation 3.1 it follows from the definition of the prediction residual $\Delta_z(t)$ that $s_z(t) - \hat{s}_z(t) = \Delta_z(t)$. Thus, the calculation of the sign of the double-resolution prediction error can be calculated as

$$\begin{cases} !\big(\tilde{s}_z(t)'\text{LSB}\big), & |\Delta_z(t)| \leq m_z(t), \\ !\big(\Delta_z(t)'\text{MSB}\big), & \text{otherwise.} \end{cases} \tag{4.1}$$

By evaluating the least significant bit (MSB) of the double-resolution predicted sample $\tilde{s}_z(t)$ it can be determined if the value is odd or even. As the signed integer values are represented using twos complement, the most significant bit (MSB) of the prediction residual $\Delta_z(t)$ gives its sign. The calculation is implemented in hardware as shown in Figure 4.3.

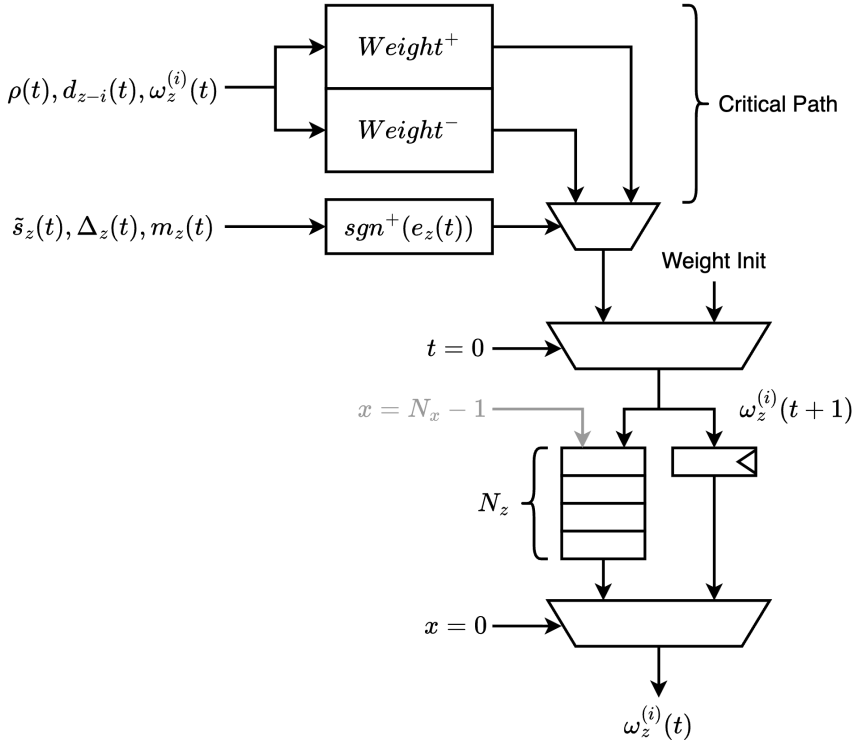

Figure 4.2: Block diagram of the weight update stage as implemented by Vorhaug. Speculative calculation of the new weights are processed in parallel with the sign of the double-resolution prediction error $e_z(t)$. A FIFO buffer of size N_z stores weight values for intermittent lines of the same band.

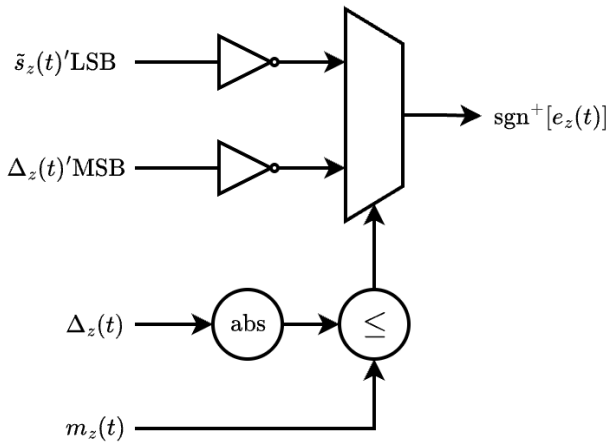

Figure 4.3: Calculation of the sign of the double-resolution prediction error as implemented by Vorhaug, following the mathematical optimization proposed by Sánchez et al.

Chapter 5

Implementation and Verification

This sections presents the proposed updates to hardware microarchitecture for reduced weight update frequency, as well as corresponding changes to the high level python model.

5.1 Modifications to High Level Model

Before choosing to implement the reduced weight update scheme in hardware, its compression performance is evaluated. This is done by modifying the high level model of the algorithm written by Vorhaug et al. The modified code is run for a set of different PAEs, ranging from 0 to 15, and the resulting compression ratios are compared to the results of the original standard given the same paramters and images. This range of PAEs is chosen based on analysis of the PSNRs for reconstructed images of the original standard, as will be presented in chapter 6.

Code listing 5.1: Pseudocode for updated Python model weight update step.

```
for all z and t:
  if t = 0:
    continue
  if t = 1:
    weight(z, t) = init_weight(t)

  if x < 3: # keep same weight
    weight(z, t) = weight(z, t-1)
  else: # apply weight update
    new_weight = weight(z, t-4) + weight_offset(z, t-4)

    if x > 3: # refine weight
      new_weight = new_weight + weight(z, t-1)
      new_weight = new_weight // 2

    weight(z, t) = clip(new_weight)
```

Code listing 5.1 shows pseudocode for the modified weight update scheme of the high level model, following the method described in Equation 3.2. Weights are updated using the weight from four samples earlier in the processing order, with weight refinements being applied using the weight of the previous sample.

5.2 CNES Provided Software

The Centre National d'Études Spatiales (CNES) has, as part of the work of the CCSDS, developed software tools for working with the issue 2 algorithm. Runnable binaries for these tools are available online, including an encoder, decoder and a tool for generating parameter headers. These tools are

used during the development for verification of the calculations of the python model and results of the hardware implementation. Additionally the decoder is used to decode the compressed images.

5.3 Reduced Weight Updates

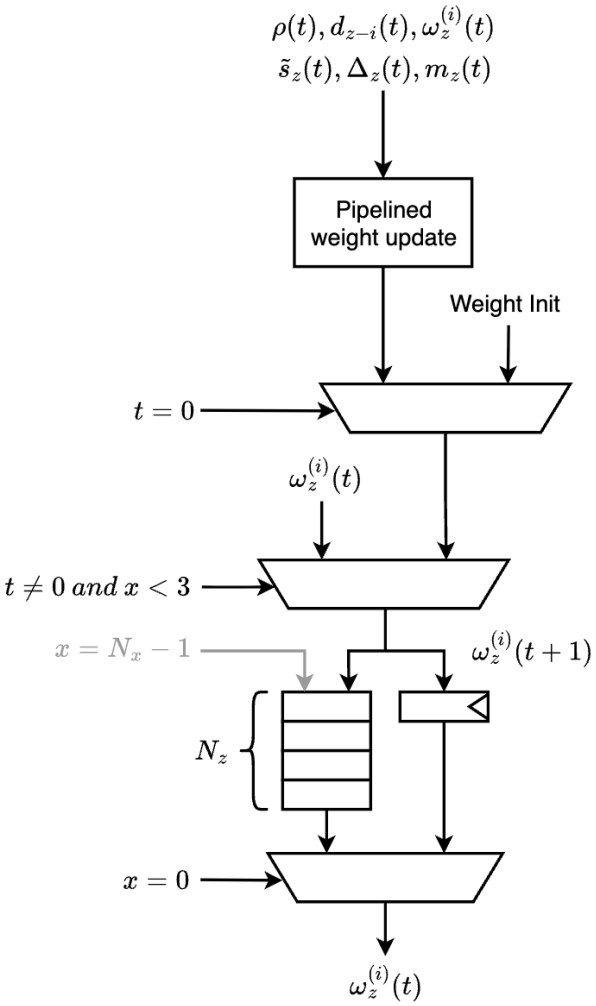

Figure 5.1: Proposed block diagram for the reduced weight updating scheme with weight refinements.

Figure 5.1 shows the proposed changes to the weight update stage in the design by Vorhaug. Since delayed weight updates are only applied after the first four samples of each line, the FIFO buffer solution remains unchanged. An additional MUX is added to allow cycling the same weight back for reuse in the first three samples of each line. The speculative blocks of the old design are replaced with a pipelined version shown in Figure 5.2. Pipeline registers are inserted between the major computational steps. The first three stages calculate the weight offset $\omega_{z,\text{offset}}^{(i)}(t)$ as per Equation 2.13. The fourth stage adds the weight offset to the old weight $\omega_z^{(i)}(t)$, producing the unrefined, updated weight. The final stage performs the weight refinement, requiring the weight values from the previous sample.

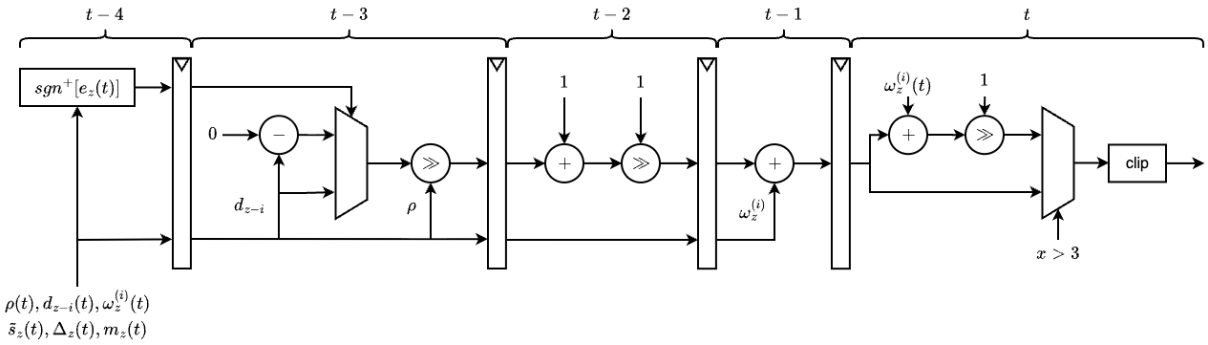

Figure 5.2: Pipelined version of the weight update calculation. The first four stages use values from four cycles earlier in the sample ordering so that calculation can start early. In the final stage the weight of the current sample t is needed to produce the updated weight for sample $t+1$.

In every cycle the set of parameters needed for weight calculation are passed to the weight update pipeline. These include the weight update scaling exponent $\rho(t)$, the central local difference $d_{z-i}(t)$ of the corresponding prediction band i, the current weight value $\omega_z^{(i)}(t)$, the double-resolution prediction sample $\tilde{s}_z(t)$, the prediction residual $\Delta_z(t)$ and the maxmum error $m_z(t)$. These values propagate through the pipeline along with the intermediary calculated weight update value.

5.4 Verification

Verification methods are selected in order to allow easy comparisons of the results of this work with those of Jia et al. in [1]. As mentioned, a combination of absolute and relative error limits should be used to minimise PSNR. However, the work by Jia et al. only uses absolute error limits. Thus, a similar approach is taken for this work. Table 5.1 shows the algorithm parameters that are used during tests. These are supported by the hardware implementation by Vorhaug, allowing comparisons of the results from high level model with those the hardware simulations.

Table 5.1: Selected predictor and hybrid encoder parameters. Values are selected based on the work by Blanes et al., and are, except for the absolute and relative error limits, the same as those used by Vorhaug [2, 18]. Table recreated from [2].

Name	Symbol	Configuration
Sample type		Unsigned
Dynamic range	D	16
Sample encoding order		BIL
Entropy coder type		Hybrid coder
Quantizer method		Absolute
Number of prediction bands	P	3
Register size	R	64
Local sum type		Narrow column-oriented
Prediction mode		Reduced
Weight component resolution	Ω	19
Weight initialization method		Default
Weight update initial parameter	$\nu_{\min}$	-1
Weight update final parameter	$\nu_{\max}$	4
Weight update change interval	t_{inc}	2^6
Weight update exponent offsets	$\zeta_z^{(i)}, \zeta_z^*$	0
Periodic error limit update		Disabled
Band-varying absolute limits		Disabled
Absolute error limit	a_z	*Varied*
Band-varying relative limits		Disabled
Relative error limit	r_z	0
Sample representative resolution	Θ	3
Sample representative offset	ψ_z	7
Band-varying offset		Disabled
Sample representative damping	θ_z	3
Band-varying damping		Disabled
Unary length limit	$U_{\max}$	18
Rescaling counter size	γ^*	6
Initial count exponent	γ_0	1
Accumulator initialization value	$\tilde{\Sigma}_z(0)$	$4 \cdot 2^{\gamma_0}$

Chapter 6

Results

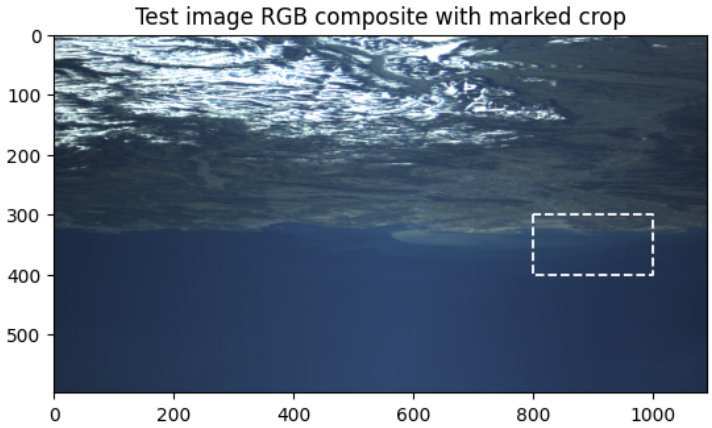

Figure 6.1: RGB composite of the HYPSO-1 image used in the following test of compression ratios and compression noise. For tests using the high level model of the algorithm the outline section of the image is used.

Figure 6.1 shows the HYPSO-1 image used for the following results. A coastline image was chosen to allow comparisons with results from the work by Jia et al. in which coastline images are used. For tests using the high level model written in python only a cropped version of the image is used. This is marked in the figure with a white outline. This is due to the memory usage of the program.

Figure 6.2 shows the compression ratios and PSNRs for a set of PAEs in the range $0 < PAE < 16$. Higher error limits increases the compression ratio while introducing compression noise. The degradation in the quality of the reconstructed image can be seen in terms of the PSNR. Visual inspection of reconstructed images and comparisons of image spectres with the original image show that the PSNR should not sink lower than around $200dB$. At PAEs over 4, the PSNR sinks under this threshold, at a CR of roughly 3.2. Here, only the absolute error limit feature of the algorithm is used, with the relative being set to zero. By also including relative limits the image PSNR would improve, at the cost of a lower compression ratio.

Figures 6.3 and 6.4 show the pixel spectrum and per band PSNR for two different PAEs: 128 and 4. The very high PAE of 128 to demonstrate the degradation of the image quality at extreme error limits, with PSNR as low as roughly $140dB$ and relative pixel value differences up to several percent. As the PAE drops to 4, the PSNR and relative pixel differences drop, reaching a PSNR over $200dB$. The relative pixel difference tends to be smallest in the spectral bands ranging from 60 to 100.

Figure 6.5 compares the compression performance for the new weight update scheme with that of the original standard. Results are gathered from the modified high level model, both including

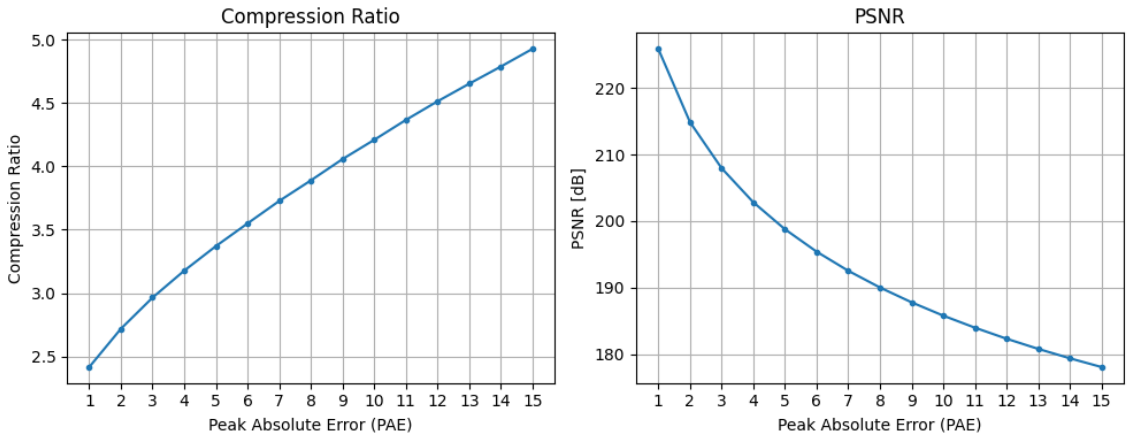

Figure 6.2: Compression ratio and PSNR for a range of peak absolute errors. As the PAE increases the compression noise increases and PSNR rapidly decreases.

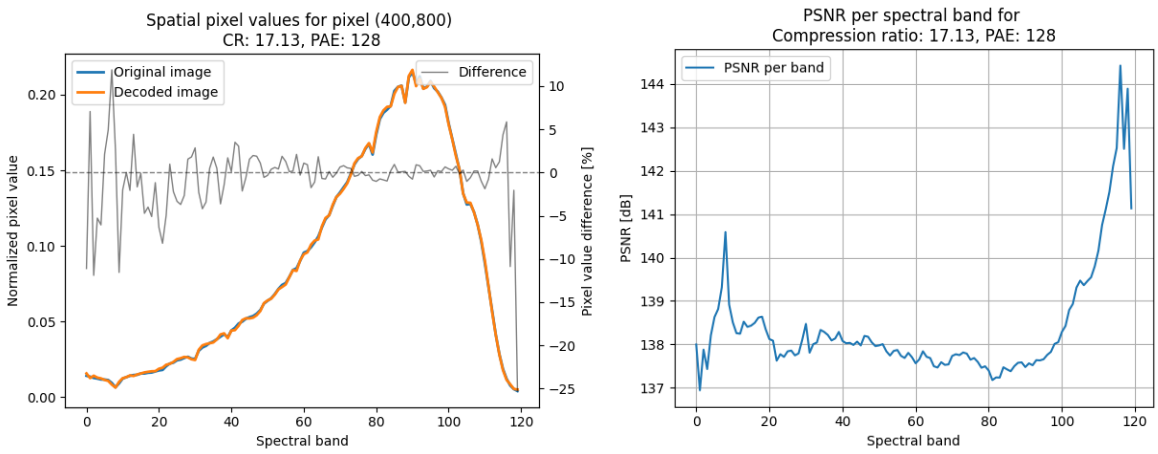

Figure 6.3: As an extreme example consider the pixel spectrum and corresponding PSNR for an image compressed with a PAE of 128. A high compression ratio of 17.13 is achieved but at a significant degradation in image quality.

without weight refinement (to the left) and with refinement (to the right). For both methods the difference in compression performance increases as the PAE is increased. With the inclusion of the weight refinement step the average reduction in compression performance across the range of PAEs is reduced from 5.24% to 3.08%. Table 6.1 summarizes the results of the compression ratios shown in the figure, comparing them to those of Jia et al.

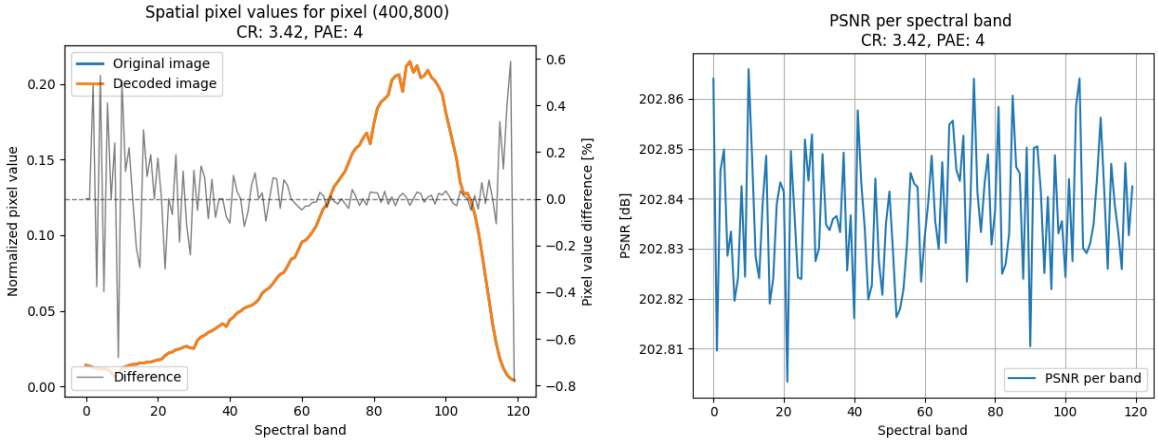

Figure 6.4: Pixel spectrum and corresponding PSNR for an image compressed with a PAE of 4.

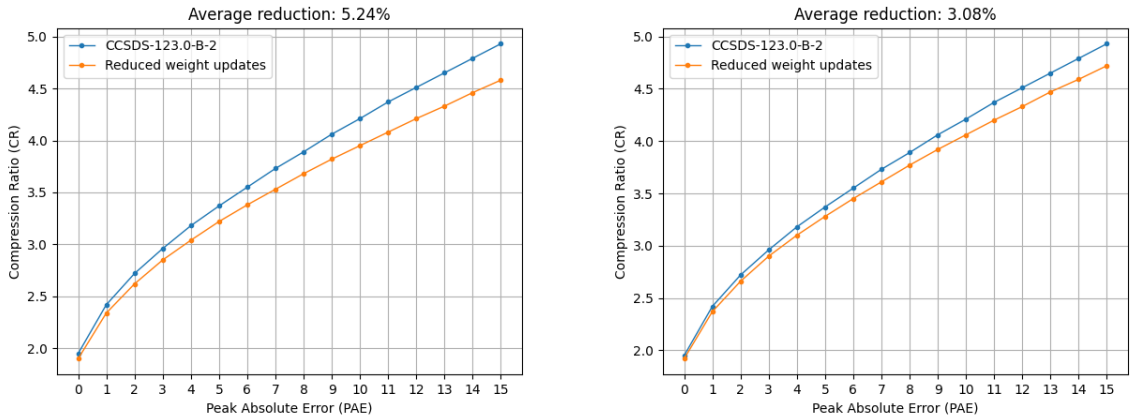

Figure 6.5: Coding performance for different peak absolute error limits for reduced weight update frequency, with and without weight refinement, as compared to the original standard. Implemented in high level python model.

Table 6.1: Reductions in compression performance for coastline image for this work and the work by Jia et al.

PAE	Coastline, Jia et al.	Coastline, this work
0	−1.53%	−1.54%
1	−2.88%	−2.07%
3	−3.86%	−2.36%
7	−4.30%	−3.22%
15	−1.23%	−4.46%

Chapter 7

Discussion and Further Work

This section discusses the preliminary results presented in the previous section, comparing them to that of other work within the field. Furthermore, future work is presented.

7.1 Compression Performance

As shown in Table 6.1, the reductions in compression ratios by introduction of the new weight update scheme in the high level model are comparable to those of Jia et al. As showed by Figure 6.2, to not introduce excessive noise in the image during compression, the PAE should be kept around 4. For the PAE 3, the reductions in compression ratios of Jia et al. and this work are 3.86% and 2.36%. The performance of the algorithm depends heavily on the images that are used, as well as the image sensor, meaning some discrepancies are to be expected. The tests performed by Jia et al. use a large selection of images from the AVIRIS hyperspectral image collection grouped by characteristics such as *coastline* (see the table). Using a larger set of images for testing the performance of the new weight update scheme should be performed in future work. However, these initial results are sufficient to justify experimentation with the method in hardware.

Results of Figure 6.5 support the addition of the weight refinement step, added by Jia et al. to remedy the loss in compression performance. Although this step requires the use of the weights of the previous sample (Equation 3.2), it does not reduce the gains in speed as weight update calculation can still begin four samples earlier in the processing order.

Weight vectors accumulate inside the confines of a spectral band, meaning that the weights of previous samples in the band are used to produce weights for subsequent samples. For the introduced weight update scheme, weight updates are not applied to the first samples of each new line (Figure 3.3 and Equation 3.2). This simplifies hardware implementation when using the BIL ordering, as no special logic is needed for the boundaries of different frames. Operands of the weight update can be passed to the pipelined weight update stage in the same order they are available. An alternative approach is to apply weight updates with refinement for all samples after $t = 3$. This could improve the performance as the sign algorithm would be provided with a higher granularity in terms of samples. Tests using this approach should be performed in the high level model to verify this. However, this would come at the cost of increased hardware complexity and resource utilization.

7.2 Hardware Implementation

The proposed pipelined weight update stage presented in Figure 5.2 has in this work not been verified in hardware. In future work, corresponding changes to the VHDL RTL of the implementation by Vorhaug should be made. After implementation, the design will be verified using the existing verification framework where intermediary calculations are compared to the results of the modified

high level model. Finally the design should be integrated for use onboard the Hypso-1 and Hypso-2 satellites.

As heavily emphasized in earlier sections, the weight update stage forms the critical path of the current design. The proposed pipeline splits the stage into its principle arithmetic operations, meaning it can no longer be split up any further. According to the work by Jia et al., this should result in a roughly three times increase in throughput. However, performance gains of the design will depend on which pipeline stage becomes the next bottleneck after the path of the weight update stage has been resolved. In future work, timing analysis should be performed to determine the new critical path.

The design by Vorhaug is the CCSDS-123.0-B-2 compliant hardware implementation with the smallest area footprint in open literature. The proposed changes remove the duplicated hardware for speculative calculation of weight updates (Figure 4.2), further reducing the area footprint. This is replaced by the pipeline of Figure 5.2, in which pipeline registers are inserted between arithmetic operations. These registers somewhat increases the area again, but not significantly. Therefore, the resource utilization of the new approach is expected to be lower than that of the current design. Analysis of resource utilization after implementation will confirm or disprove this.

7.3 Image Quality

The image data from Hypso contains inherent noise originating from the sensor, as well as signal noise caused by athmospheric effects (section 2.2). The analysis of Figure 6.2 shows the PSNR of the reconstructed images, giving insight into how the compression degrades the quality of the final image. In the selection of the PAE, this data must be taken into account. Using PSNR as the only metric for quality can be challenging as it is hard to decide an definitive value for what a good PSNR should be. Furthermore, it is important to know whether the compression noise affects the inherent noisy parts of the image or the signal which is vital for the usecases of the images. This should be analyzed in future work.

In this work, only the absolute error limits of the standard have been used during tests (Equation 2.4). The effects of the relative error limits should also be explored in terms of compression performance and PSNR.

7.4 Band-dependant Error Limits

As evident from figures 6.3 and 6.4, the algorithm better preserves the samples values of central spectral bands. This is likely caused by the higher image quality of these bands as a result of the image sensor of Hypso-1 [6, 15]. In order to exploit this feature of the images to achieve higher compression ratios, the band-dependant error limit feature of the standard should be explored. By setting higher error limits for the lower quality outer bands, which are of less interest, the compression performance would increase. The current design does not support this feature of the standard. If tests using the high level model are promising, support should be added to the hardware. This would come at the cost of additional storage to store the error limit vectors for absolute and relative limits respectively.

7.5 Image Reconstruction

As detailed in The Consultative Committee for Space Data Systems (CCSDS), 'Lossless multispectral and hyperspectral image compression,' CCSDS, Green Book Report, Issue 2 120.2-G-2, 2022, and touched upon in subsection 2.1.2, the decoder needs to able to reconstruct the sample model created by the encoder. This means recreating the sample representatives $s_z''(t)$ and predicted sample values $\hat{s}_z(t)$. In order to do this, the decoder needs to know how weight updates were performed during

compression. By changing the weight update scheme beyond that of the original standard, an issue 2 compliant decoder can no longer decode compressed images produced by the compressor. In order to verify the correct functioning of the proposed design, an issue 2 decoder must be implemented in future work. The decoder should be designed to allow easy modifications, so that further experiments with the weight update scheme can be performed. The decoder must also be written to support integration into the Hypso software stack.

Chapter 8

Conclusions

With the advent of small satellite technology, such as the HYPSO project at NTNU, hyperspectral imagery has become readily available. These image sensors produce large amounts of data which needs to be downlinked to earth using channels with limited bandwidth. This prompts the need for onboard compression using algorithms such as CCSDS-123.0-B-2. Hardware implementations for FPGA of the algorithm must balance hardware area footprint, compression performance and data throughput. Three main approaches have been proposed to alleviate the spectral and spatial data dependencies that make these requirements hard to fulfill. Báscones et al. suggest a careful rearranging of the input samples, giving data dependencies time to resolve and allowing deep pipelining. Chatziantoniou et al. uses pre-quantization, at the loss of some compression performance, to remove the in-loop quantization and sample representatives. Both of these approaches reach a throughput of $285 Msamples/s$ using a single core, with the possibility of reaching higher performance with image segmentation and multicore designs. A better approach for hardware resource constrained systems is the mathematical optimization approach combined with reduced weight updating frequency, proposed by Sánchez et al. and Jia et al. respectively. The latter presents a predictor with a throughput of $384 Msamples/s$ with a small hardware footprint of only 3995 LUTs and 5050 FFs.

This work analyses the compression performance of the reduced weight updating scheme proposed by Jia et al. using a modified version of the high level model of the algorithm written by Vorhaug et al. Preliminary results show a decrease in compression performance of *3.08%*, averaged across a range of absolute error limits range from *0* to *15*. These results are comparable to those of Jia et al., motivating hardware implementation of the approach.

For use onboard the HYPSO project, changes to the hardware implementation by Vorhaug are proposed to support the new weight updating scheme. The solution is expected to increase the maximum clock frequency by a factor of three, while reducing the hardware resource utilization. Future work should implement the changes and verify these claims. Furthermore, software for decoding images compressed using the reduced weight update frequency scheme must be implemented.

Bibliography

[1] L. Jia, Q. Wang, L. Zhang, C. Song and P. Zhang, 'Removal of the Feedback Loop in CCSDS 123.0-B-2 during Hardware Implementation,' *IEEE Geoscience and Remote Sensing Letters*, 2025, ISSN: 1545-598X, 1558-0571. DOI: 10.1109/LGRS.2025.3601194.

[2] D. Vorhaug, 'Hyperspectral Image Compression Accelerator On FPGA Using CCSDS 123.0-B-2,' M.S. thesis, NTNU, 2024.

[3] E. J. Ientilucci and S. Adler-Golden, 'Atmospheric Compensation of Hyperspectral Data: An Overview and Review of In-Scene and Physics-Based Approaches,' *IEEE Geoscience and Remote Sensing Magazine*, vol. 7, no. 2, Jun. 2019, ISSN: 2168-6831. DOI: 10.1109/MGRS.2019.2904706.

[4] M. Parente, J. Kerekes and R. Heylen, 'A Special Issue on Hyperspectral Imaging [From the Guest Editors],' *IEEE Geoscience and Remote Sensing Magazine*, vol. 7, no. 2, Jun. 2019, ISSN: 2168-6831. DOI: 10.1109/MGRS.2019.2912617.

[5] M. E. Grøtte, R. Birkeland, E. Honoré-Livermore, S. Bakken, J. L. Garrett, E. F. Prentice, F. Sigernes, M. Orlandić, J. T. Gravdahl and T. A. Johansen, 'Ocean Color Hyperspectral Remote Sensing With High Resolution and Low Latency—The HYPSO-1 CubeSat Mission,' *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, 2022, ISSN: 1558-0644. DOI: 10.1109/TGRS.2021.3080175.

[6] D. Blondeau-Patissier, J. F. R. Gower, A. G. Dekker, S. R. Phinn and V. E. Brando, 'A review of ocean color remote sensing methods and statistical techniques for the detection, mapping and analysis of phytoplankton blooms in coastal and open oceans,' *Progress in Oceanography*, vol. 123, 1st Apr. 2014, ISSN: 0079-6611. DOI: 10.1016/j.pocean.2013.12.008.

[7] M. Hernández-Cabronero, A. B. Kiely, M. Klimesh, I. Blanes, J. Ligo, E. Magli and J. Serra-Sagristà, 'The CCSDS 123.0-B-2 "Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression" Standard: A comprehensive review,' *IEEE Geoscience and Remote Sensing Magazine*, vol. 9, no. 4, Dec. 2021, ISSN: 2168-6831. DOI: 10.1109/MGRS.2020.3048443.

[8] The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

[9] M. Klimesh, 'Low-complexity lossless compression of hyperspectral imagery via adaptive filtering,' *Interplanetary Network Progress Report*, Jan. 2005.

[10] D. Keymeulen, S. Shin, J. Riddley, M. Klimesh, A. Kiely, E. Liggett, P. Sullivan, M. Bernas, H. Ghossemi, G. Flesch, M. Cheng, S. Dolinar, D. Dolman, K. Roth, C. Holyoake, K. Crocker and A. Smith, 'High performance space computing with system-on-chip instrument avionics for space-based next generation imaging spectrometers (ngis),' Aug. 2018, pp. 33–36. DOI: 10.1109/AHS.2018.8541473.

[11] The Consultative Committee for Space Data Systems (CCSDS), 'Lossless multispectral and hyperspectral image compression,' CCSDS, Green Book Report, Issue 2 120.2-G-2, 2022.

[12] A. Gersho, 'Adaptive filtering with binary reinforcement,' *IEEE Transactions on Information Theory*, vol. 30, no. 2, pp. 191–199, Mar. 1984, ISSN: 1557-9654. DOI: 10.1109/TIT.1984.1056890. Accessed: 5th Dec. 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1056890.

[13] M. A. Rahman and M. Hamada, 'Lossless Image Compression Techniques: A State-of-the-Art Survey,' *Symmetry*, vol. 11, no. 10, p. 1274, Oct. 2019, ISSN: 2073-8994. DOI: 10.3390/sym11101274. Accessed: 4th Dec. 2025. [Online]. Available: https://www.mdpi.com/2073-8994/11/10/1274.

[14] P. Chatziantoniou, N. Kranitis and A. Tsigkanos, 'A High-Performance RTL Implementation of the CCSDS-123.0-B-2 Hybrid Entropy Coder on a space-grade SRAM FPGA,' in *OBPDC*, 2020.

[15] S. Bakken, M. B. Henriksen, R. Birkeland, D. D. Langer, A. E. Oudijk, S. Berg, Y. Pursley, J. L. Garrett, F. Gran-Jansen, E. Honoré-Livermore, M. E. Grøtte, B. A. Kristiansen, M. Orlandic, P. Gader, A. J. Sørensen, F. Sigernes, G. Johnsen and T. A. Johansen, 'HYPSO-1 CubeSat: First Images and In-Orbit Characterization,' *Remote Sensing*, vol. 15, no. 3, Jan. 2023, ISSN: 2072-4292. DOI: 10.3390/rs15030755.

[16] D. Báscones, C. Gonzalez and D. Mozos, 'A Real-Time FPGA Implementation of the CCSDS 123.0-B-2 Standard,' *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, 2022, ISSN: 1558-0644. DOI: 10.1109/TGRS.2022.3160646.

[17] A. Sánchez, I. Blanes, Y. Barrios, M. Hernández-Cabronero, J. Bartrina-Rapesta, J. Serra-Sagristà and R. Sarmiento, 'Reducing Data Dependencies in the Feedback Loop of the CCSDS 123.0-B-2 Predictor,' *IEEE Geoscience and Remote Sensing Letters*, vol. 19, 2022, ISSN: 1558-0571. DOI: 10.1109/LGRS.2022.3213975.

[18] I. Blanes, A. Kiely, M. Hernández-Cabronero and J. Serra-Sagristà, 'Performance Impact of Parameter Tuning on the CCSDS-123.0-B-2 Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression Standard,' *Remote Sensing*, vol. 11, no. 11, Jan. 2019, ISSN: 2072-4292. DOI: 10.3390/rs11111390.

[19] D. Salomon, *Data Compression the Complete Reference*, 3rd ed. New York: Springer, 2004, ISBN: 978-0-387-40697-8 978-0-387-21832-8.

[20] J. Lamoureux and W. Luk, 'An Overview of Low-Power Techniques for Field-Programmable Gate Arrays,' in *2008 NASA/ESA Conference on Adaptive Hardware and Systems*, Jun. 2008. DOI: 10.1109/AHS.2008.71.

[21] M. Wolf, 'Chapter 10 - embedded multiprocessors,' in *Computers as Components (Fifth Edition)*, ser. The Morgan Kaufmann Series in Computer Architecture and Design, M. Wolf, Ed., Fifth Edition, Morgan Kaufmann, 2023, pp. 453–484. DOI: 10.1016/B978-0-323-85128-2.00010-4.

[22] R. Sarpeshkar, 'Universal Principles for Ultra Low Power and Energy Efficient Design,' *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 4, Apr. 2012, ISSN: 1558-3791. DOI: 10.1109/TCSII.2012.2188451.

[23] Y. Barrios, A. Sánchez, R. Guerra and R. Sarmiento, 'Hardware Implementation of the CCSDS 123.0-B-2 Near-Lossless Compression Standard Following an HLS Design Methodology,' *Remote Sensing*, vol. 13, no. 21, p. 4388, Jan. 2021, ISSN: 2072-4292. DOI: 10.3390/rs13214388.

[24] A. Tsigkanos, N. Kranitis, D. Theodoropoulos and A. Paschalis, 'High-Performance COTS FPGA SoC for Parallel Hyperspectral Image Compression With CCSDS-123.0-B-1,' *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 11, Nov. 2020, ISSN: 1557-9999. DOI: 10.1109/TVLSI.2020.3020164.

[25] M. Orlandic, J. A. Fjeldtvedt and T. A. Johansen, 'A Parallel FPGA Implementation of the CCSDS-123 Compression Algorithm,' *11*, 2019, ISSN: 2072-4292. DOI: 10.3390/rs11060673.

[26] P. Chatziantoniou, A. Paschalis, N. Kranitis, A. Tsigkanos and D. Theodoropoulos, 'Scalable Data-Rate Near-Lossless Hyperspectral Image Compression Based on Parallel Implementation of CCSDS-123.0-B-2,' in *OBPDC*, 2022. DOI: 10.5281/zenodo.7269802.

[27] Y. Barrios, J. Bartrina-Rapestà, M. Hernández-Cabronero, A. Sánchez, I. Blanes, J. Serra-Sagrista and R. Sarmiento, 'Removing Data Dependencies in the CCSDS 123.0-B-2 Predictor Weight Updating,' *IEEE Geoscience and Remote Sensing Letters*, vol. 21, 2024, ISSN: 1558-0571. DOI: 10.1109/LGRS.2024.3362376.

[28] D. Vorhaug, S. Boyle and M. Orlandić, 'High-Level CCSDS 123.0-B-2 Hyperspectral Image Compressor Verification Model,' in *2024 14th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing (WHISPERS)*, Dec. 2024. DOI: 10.1109/WHISPERS65427.2024.10876459.